

Credit Card Approval Prediction

INTRODUCTION

Banks and financial institutions receive thousands of credit card applications every day from customers belonging to different financial backgrounds. Evaluating each application manually requires significant time and effort, making the approval process slow and prone to inconsistencies. Factors such as applicant income, employment history, family status, housing conditions, credit amount, and previous credit bureau inquiries all influence the final approval decision. Traditional manual assessment methods increase operational workload and may lead to delays in customer service and inconsistent decision-making.

The **Credit Card Approval Prediction** project addresses these challenges by applying Machine Learning techniques to automate the preliminary credit card eligibility assessment process. The system analyzes applicant demographic, financial, employment, and credit-related information to determine whether an applicant is likely to be eligible for a credit card. Instead of relying solely on manual verification, the application provides an instant prediction supported by a trained machine learning model.

The project follows a complete machine learning workflow beginning with dataset collection, exploratory data analysis, preprocessing, feature engineering, model training, evaluation, model selection, deployment, and web application development. Multiple supervised learning algorithms including Logistic Regression, Decision Tree, Random Forest, and XGBoost were trained and evaluated using the prepared dataset. Based on performance metrics such as Accuracy, Precision, Recall, F1-Score, Balanced Accuracy, and ROC-AUC, XGBoost was selected as the final prediction model.

The trained model is integrated into a Flask-based web application that enables users to enter applicant details through an intuitive interface and instantly receive an eligibility prediction. The application also provides a confidence score, eligibility level, and personalized recommendations, helping users understand the prediction results more effectively.

The system demonstrates how machine learning can support banking operations by improving decision-making speed, reducing manual effort, and providing consistent eligibility assessments while maintaining a user-friendly experience.

PROJECT DESCRIPTION

Problem Statement

Banks process a large number of credit card applications daily. Manual verification of applicant information is time-consuming, resource-intensive, and susceptible to inconsistencies. Applicants often remain uncertain about their eligibility before submitting an application, leading to unnecessary rejections and increased processing workload for financial institutions.

The absence of an intelligent pre-screening system affects both customers and banks. Customers may experience delays or apply without understanding their eligibility, while financial institutions spend additional resources reviewing applications that do not satisfy the required criteria.

Therefore, there is a need for an automated system capable of analyzing applicant information and predicting credit card eligibility quickly and accurately using Machine Learning.

Proposed Solution

The proposed solution is a Machine Learning-based web application that predicts credit card approval

eligibility using applicant demographic, financial, employment, and credit bureau information.

The system performs automatic preprocessing of user inputs, including categorical encoding, numerical scaling, and feature transformation, before passing the processed data to the trained machine learning model.

After prediction, the application displays:

- Eligibility Status
- Confidence Score
- Eligibility Level
- Personalized Recommendations

The application is built using Flask for the backend and HTML, CSS, and JavaScript for the frontend, providing users with a responsive and easy-to-use interface.

Objectives

The primary objectives of the project are:

- To automate the preliminary credit card eligibility assessment process.
 - To reduce manual effort involved in application screening.
 - To improve consistency and accuracy in eligibility prediction.
 - To compare multiple machine learning algorithms and select the best-performing model.
 - To develop a responsive Flask web application for real-time predictions.
 - To provide applicants with confidence scores and personalized recommendations.
 - To deploy the application on the cloud for easy accessibility.
-

Scope of the Project

The scope of the project includes:

- Collection and preparation of applicant data.
- Data preprocessing and feature engineering.
- Exploratory Data Analysis (EDA).
- Training multiple classification models.
- Model evaluation and comparison.
- Selection of the best-performing machine learning model.
- Integration of the model with a Flask web application.
- Deployment using Render.
- Documentation and project demonstration.

The project focuses on providing a preliminary eligibility assessment and does not replace the final verification process performed by financial institutions.

Project Features

The system provides the following major features:

- Machine Learning-based credit card eligibility prediction.
- Automated preprocessing pipeline.
- Feature engineering for improved model performance.

- Comparison of Logistic Regression, Decision Tree, Random Forest, and XGBoost.
 - Real-time prediction through a Flask web application.
 - Responsive user interface.
 - Client-side and server-side input validation.
 - Confidence score calculation.
 - Eligibility level classification.
 - Personalized financial recommendations.
 - Cloud deployment using Render.
-

REAL-WORLD SCENARIOS

Scenario 1 – Customer Eligibility Assessment

A customer planning to apply for a credit card accesses the application to check their eligibility before submitting an official application. By entering demographic and financial information, the customer receives an instant prediction that helps determine whether they are likely to qualify for a credit card.

Scenario 2 – Bank Credit Analyst

A bank credit analyst uses the application to perform preliminary screening of multiple applicants. The prediction system identifies potentially eligible applicants, allowing analysts to focus on detailed verification for shortlisted applications.

Scenario 3 – Financial Institutions

Financial institutions can integrate the prediction system into their internal workflows to automate the first stage of credit card application screening, reducing manual effort and improving operational efficiency.

Scenario 4 – Self-Service Eligibility Checker

Prospective customers use the web application to evaluate their eligibility before formally applying for a credit card, reducing unnecessary applications and improving customer awareness.

Scenario 5 – Academic and Research Applications

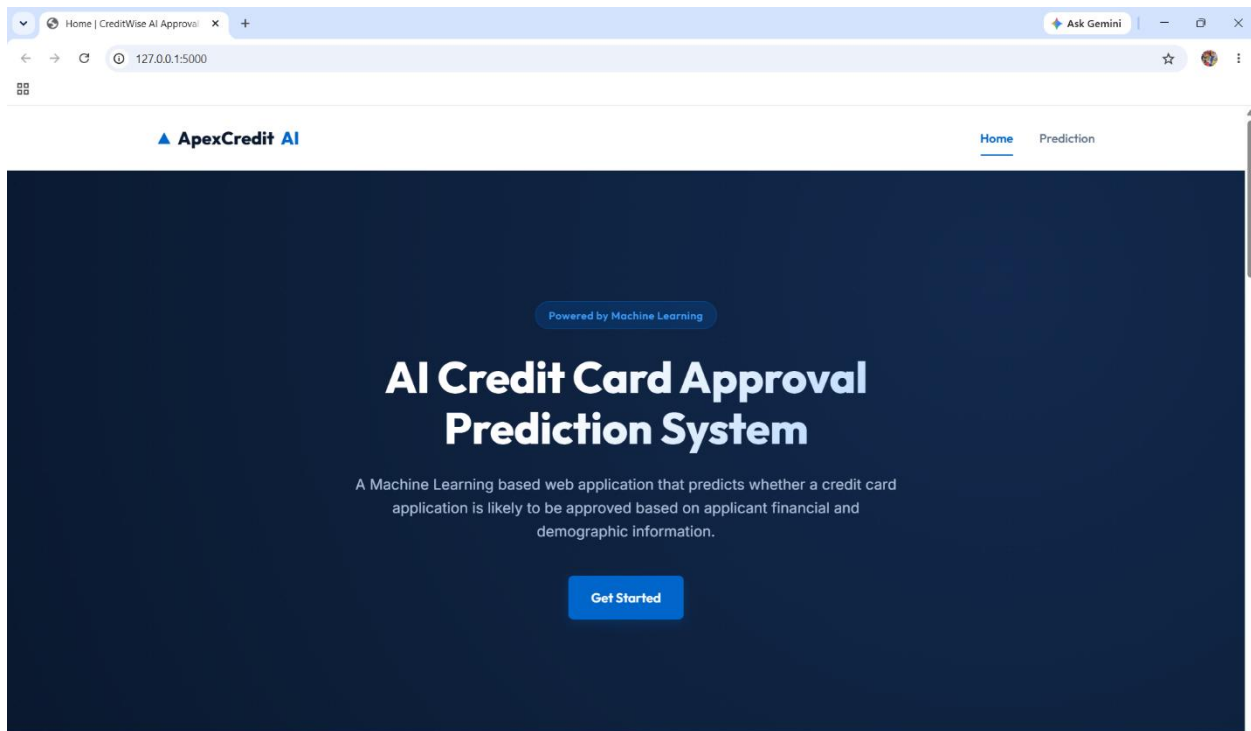
Students and researchers can use the project to study machine learning techniques, compare classification algorithms, understand preprocessing pipelines, and explore real-world financial prediction systems.

SYSTEM ARCHITECTURE

The Credit Card Approval Prediction system follows a layered architecture consisting of the Presentation Layer, Application Layer, Machine Learning Layer, and Data Layer.

Presentation Layer

The presentation layer provides the user interface developed using HTML, CSS, JavaScript, and Flask templates. Users interact with the system by entering applicant details and viewing prediction results.



Application Layer

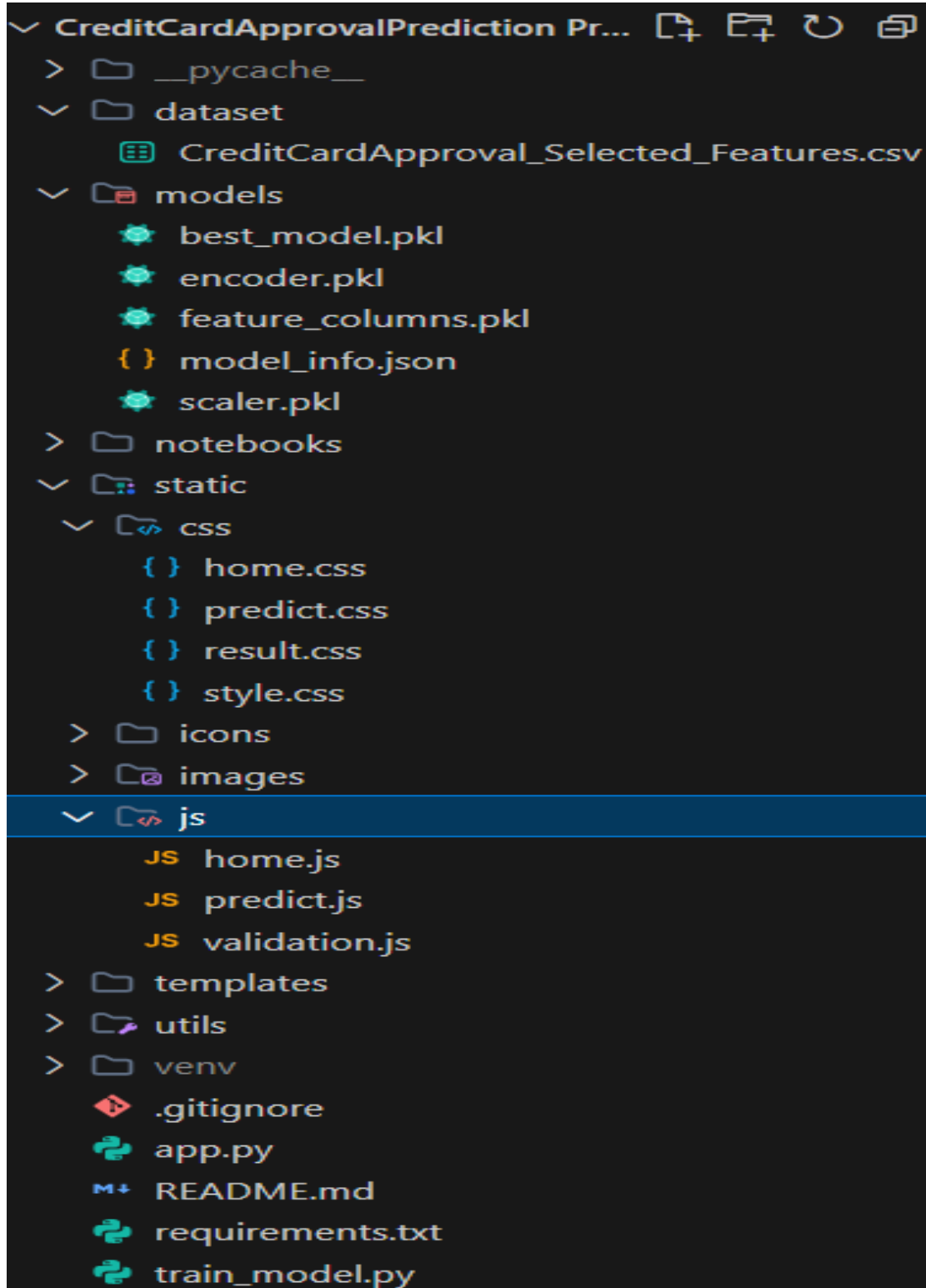
The Flask backend receives user requests, validates the submitted information, preprocesses the input data, loads the trained model, performs prediction, and returns the results to the frontend.

Machine Learning Layer

This layer contains the preprocessing pipeline, trained XGBoost model, scaler, encoder, feature engineering functions, and prediction logic.

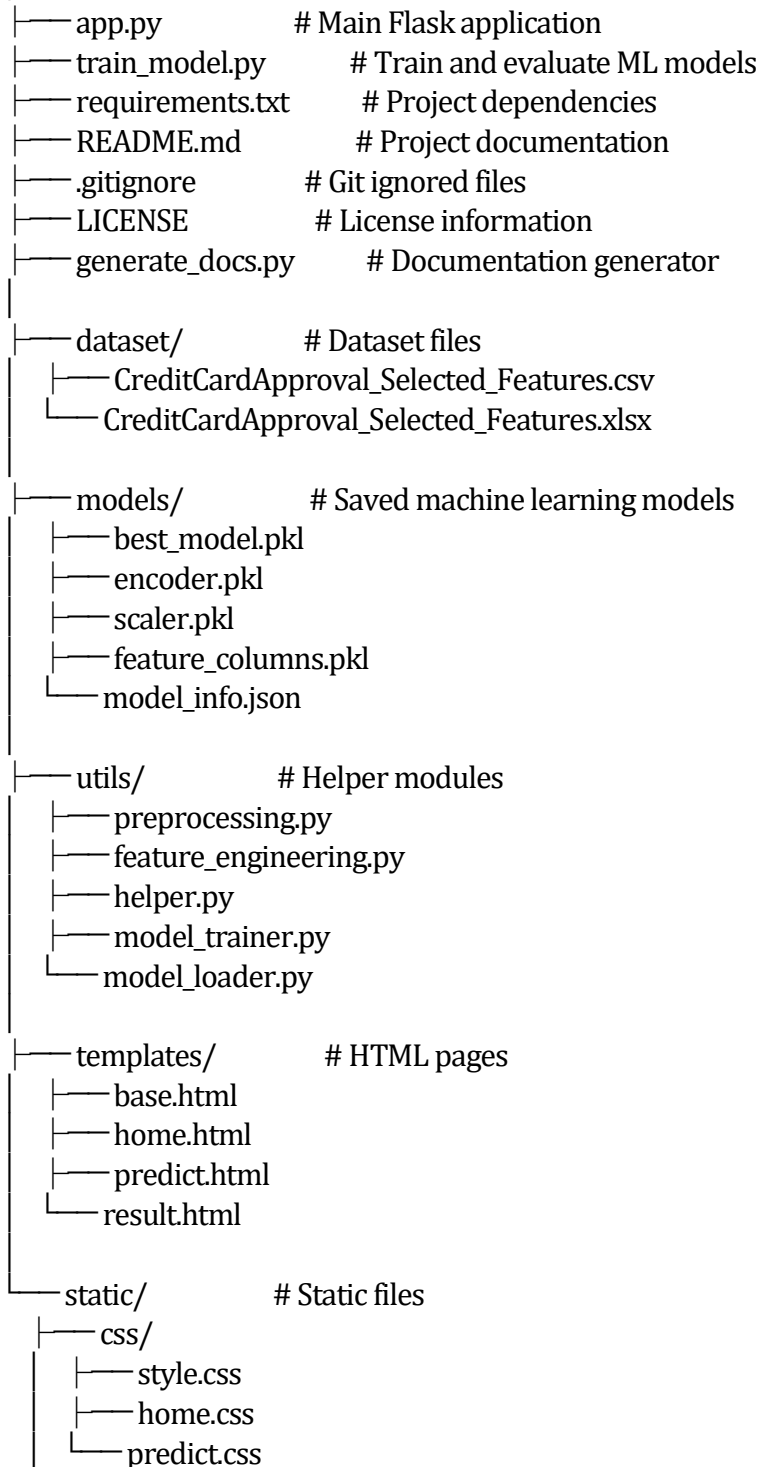
Data Layer

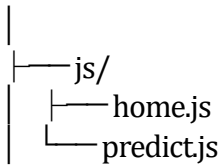
The data layer consists of the training dataset stored in CSV format and the serialized model files (best_model.pkl, scaler.pkl, encoder.pkl, and feature_columns.pkl) used during prediction.



Project Structure

CreditCardApprovalPrediction/





TECHNOLOGY STACK

Component	Technology
Programming Language	Python
Backend Framework	Flask
Frontend	HTML5, CSS3, JavaScript
Machine Learning	Scikit-learn, XGBoost
Data Processing	Pandas, NumPy
Visualization	Matplotlib, Seaborn
Model Serialization	Joblib
Dataset Format	CSV
Version Control	Git & GitHub
Deployment	Render
Production Server	Gunicorn

MILESTONE 1

Environment Setup and Dataset Preparation

The first milestone focuses on establishing the complete development environment and preparing the dataset required for building the Credit Card Approval Prediction system. A proper development environment ensures that all required libraries, frameworks, and dependencies are correctly configured before starting implementation.

The project was developed using Python as the primary programming language, Flask for backend web development, and several machine learning libraries including Scikit-learn, XGBoost, Pandas, NumPy, Matplotlib, and Seaborn. Git and GitHub were used for version control, while Render was selected for cloud deployment.

After configuring the development environment, the credit card approval dataset was collected and analyzed. Only the most relevant features required for prediction were selected to improve model performance and simplify the prediction form presented to the user.

Activity 1.1

Development Environment Setup

The first activity involved configuring the software development environment. A Python virtual

environment was created to isolate project dependencies and avoid conflicts with system-installed packages.

The following software and tools were installed:

- Python 3.10+
- Visual Studio Code
- Git
- Flask
- Scikit-learn
- XGBoost
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Joblib
- OpenPyXL
- Unicorn

A virtual environment named **venv** was created using Python's built-in virtual environment module. All required dependencies were installed using the **requirements.txt** file.

This isolated environment ensures reproducibility and simplifies project setup for other developers.

Activity 1.2

Dataset Collection

The Credit Card Approval Prediction dataset was collected from Kaggle. The dataset contains demographic, employment, financial, and credit-related information for a large number of applicants.

Initially, the dataset contained a large number of features, many of which were unnecessary for the prediction system. Therefore, feature selection was performed to retain only the attributes that significantly contribute to predicting credit card eligibility.

Examples of selected features include:

- Gender
- Own Car
- Own Realty
- Number of Children
- Family Members
- Age
- Employment Years
- Annual Income
- Credit Amount
- Loan Annuity
- Income Type
- Education Level
- Family Status

- Housing Type
- Organization Type
- Region Rating
- Credit Bureau Enquiries

The prepared dataset was saved as:

CreditCardApproval_Selected_Features.csv

This optimized dataset was used throughout preprocessing, model training, evaluation, and deployment.

Activity 1.3

Project Folder Organization

To improve maintainability and modularity, the project was organized into separate folders based on functionality.

The project structure consists of:

- **dataset/** – Stores the training dataset.
- **models/** – Contains serialized machine learning models, encoders, scalers, and metadata.
- **templates/** – Contains Flask HTML templates.
- **static/** – Stores CSS, JavaScript, images, and visualization outputs.
- **utils/** – Contains preprocessing, feature engineering, helper functions, model training, and model loading modules.

The root directory contains:

- app.py
- train_model.py
- requirements.txt
- README.md
- LICENSE
- .gitignore

This modular structure improves readability, simplifies maintenance, and supports future project enhancements.

Activity 1.4

Dependency Management

Managing project dependencies is an important aspect of software development. All required Python packages were maintained using a **requirements.txt** file.

The dependency file includes libraries such as:

- Flask
- Pandas
- NumPy
- Scikit-learn
- XGBoost
- Matplotlib
- Seaborn

- Joblib
- OpenPyXL
- Unicorn

Using a requirements file enables any team member or evaluator to recreate the project environment by executing:

```
pip install -r requirements.txt
```

This ensures consistency across different development environments and simplifies deployment.

Milestone 1 Summary

At the end of Milestone 1, the project development environment was successfully configured, the required dataset was collected and prepared, unnecessary features were removed, the project folder structure was organized, and all required dependencies were installed.

The completion of this milestone established the foundation for subsequent activities, including exploratory data analysis, preprocessing, machine learning model development, and deployment.

Milestone 1 Outcomes

- Development environment configured successfully.
- Virtual environment created.
- Required Python libraries installed.
- Dataset collected and analyzed.
- Relevant features selected.
- Project folder structure organized.
- Dependency management completed.
- Version control initialized using Git and GitHub.

MILESTONE 2

Data Analysis and Data Preprocessing

After successfully setting up the development environment and preparing the dataset, the next phase focused on understanding, analyzing, and preparing the data for machine learning. Since the quality of input data directly affects the performance of machine learning models, several preprocessing techniques were applied to ensure that the dataset was clean, consistent, and suitable for training.

This milestone included dataset exploration, descriptive statistics, univariate analysis, multivariate analysis, duplicate removal, missing value treatment, feature engineering, categorical encoding, feature scaling, and train-test splitting.

The dataset was carefully analyzed to understand the characteristics of applicant information, identify relationships among features, and prepare meaningful inputs for the prediction model.

Activity 2.1

Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to gain a deeper understanding of the dataset before applying machine learning algorithms. The analysis helped identify feature distributions, missing values, categorical variables, and relationships between different applicant attributes.

Initially, the dataset was loaded into a Pandas DataFrame using the `read_csv()` function. Several built-in

functions such as `head()`, `info()`, `shape`, and `describe()` were used to inspect the structure and characteristics of the dataset.

The following information was analyzed:

- Number of records
- Number of features
- Data types
- Missing values
- Statistical summaries
- Class distribution
- Target variable balance

This initial analysis provided valuable insights into the dataset and helped determine the preprocessing steps required before model training.

Univariate Analysis

Univariate analysis was performed to examine each feature independently.

Categorical variables such as:

- Gender
- Income Type
- Education Level
- Family Status
- Housing Type
- Organization Type

were analyzed using frequency counts and count plots.

Numerical variables such as:

- Annual Income
- Credit Amount
- Loan Annuity
- Employment Years
- Age
- Number of Children

were visualized using histograms and distribution plots.

The generated visualizations helped identify dominant categories, feature distributions, and potential outliers.

Descriptive Statistical Analysis

Descriptive statistics were generated using the Pandas `describe()` function.

The statistical analysis included:

- Count
- Mean
- Standard Deviation
- Minimum Value

- Maximum Value
- Quartiles
- Median

These statistics helped understand the spread and distribution of numerical features while also identifying unusual values that required preprocessing.

Multivariate Analysis

To understand relationships among multiple variables, correlation analysis was performed.

A correlation matrix was generated using:

`df.corr()`

The correlation values were visualized using a heatmap created with Seaborn.

The heatmap highlighted:

- Positive correlations
- Negative correlations
- Strong feature relationships
- Weak feature relationships

This analysis assisted in feature selection and helped reduce redundancy among variables.

Activity 2.2

Data Cleaning

Raw datasets often contain duplicate records, missing values, and inconsistent information. Therefore, several data cleaning techniques were applied before model development.

Duplicate Removal

Duplicate records were identified and removed using the Pandas `drop_duplicates()` function.

Removing duplicate applicant records prevents biased learning and improves overall model reliability.

Missing Value Treatment

The dataset contained missing values in a few numerical features.

Instead of removing these records, missing numerical values were replaced using the median value of the respective columns.

Median imputation was selected because it is less sensitive to outliers than the mean and helps preserve the overall data distribution.

After preprocessing, the dataset no longer contained significant missing values that could affect model performance.

Feature Engineering

Feature engineering was performed to improve the quality of information provided to the machine learning model.

New meaningful features were created from existing columns.

Examples include:

- Conversion of applicant age from days to years.

- Conversion of employment duration from days to years.
- Selection of only the most informative attributes.
- Removal of unnecessary columns.
- Preparation of numerical representations suitable for machine learning.

These transformations simplified interpretation while improving prediction quality.

Categorical Encoding

Machine learning algorithms cannot directly process text values.

Therefore, categorical attributes such as:

- Gender
- Income Type
- Education Level
- Family Status
- Housing Type
- Organization Type

were converted into numerical values using encoding techniques.

The fitted encoders were saved and later reused during prediction to ensure consistency between training and deployment.

Feature Scaling

Since numerical features have different value ranges, feature scaling was performed using **StandardScaler**.

Scaling ensured that all numerical features contributed equally during model training and prevented features with larger magnitudes from dominating the learning process.

The fitted scaler was serialized and stored for use during prediction.

Train-Test Split

The processed dataset was divided into training and testing subsets.

The data was split using an 80:20 ratio.

- Training Dataset: 80%
- Testing Dataset: 20%

The split ensured that the model could be evaluated on unseen data and reduced the risk of overfitting.

Preprocessing Pipeline

The complete preprocessing pipeline consisted of the following steps:

1. Dataset loading.
2. Duplicate removal.
3. Missing value treatment.
4. Feature engineering.
5. Categorical encoding.
6. Feature scaling.

7. Train-test splitting.
8. Serialization of scaler and encoders.

This pipeline ensured that identical preprocessing steps were applied during both training and real-time prediction.

Milestone 2 Summary

During this milestone, the dataset was thoroughly analyzed and prepared for machine learning. Exploratory data analysis provided valuable insights into applicant information, while preprocessing techniques improved data quality and consistency.

The implementation of duplicate removal, missing value handling, feature engineering, categorical encoding, and feature scaling ensured that the dataset was suitable for model training.

The completion of this milestone established a reliable preprocessing pipeline that serves as the foundation for accurate machine learning predictions.

Milestone 2 Outcomes

- Dataset explored successfully.
- Descriptive statistical analysis completed.
- Univariate analysis performed.
- Multivariate analysis completed.
- Correlation analysis generated.
- Duplicate records removed.
- Missing values handled.
- Feature engineering completed.
- Categorical variables encoded.
- Numerical features standardized.
- Training and testing datasets created.
- Scaler and encoder objects serialized for deployment.

MILESTONE 3

Machine Learning Model Development

After completing data preprocessing and feature engineering, the cleaned dataset was used to develop machine learning models capable of predicting credit card approval eligibility. Since the objective of the project is to perform binary classification (Eligible / Not Eligible), several supervised machine learning algorithms were trained and evaluated.

Different algorithms exhibit different learning capabilities depending on the dataset characteristics. Therefore, instead of relying on a single algorithm, four classification models were implemented and compared to identify the most suitable model for deployment.

The models implemented in this project include:

- Logistic Regression
- Decision Tree
- Random Forest
- XGBoost

Each model was trained using the preprocessed training dataset and evaluated using multiple performance metrics. The best-performing model was then selected and integrated into the Flask web

application.

Activity 3.1

Logistic Regression Model

Logistic Regression is one of the most widely used supervised learning algorithms for binary classification problems. It predicts the probability of an applicant belonging to one of two classes by applying the logistic (sigmoid) function.

The Logistic Regression model was trained using the standardized training dataset. During training, the algorithm learned the relationship between applicant features and the approval status.

The model was evaluated using:

- Accuracy
- Precision
- Recall
- F1-Score
- Confusion Matrix
- Balanced Accuracy
- ROC-AUC Score

Although Logistic Regression provided a strong baseline, it struggled to correctly identify minority class samples due to the imbalance present in the dataset.

Decision Tree Model

Decision Tree is a rule-based supervised learning algorithm that classifies data by repeatedly splitting the dataset based on feature values.

The Decision Tree model generated a hierarchical decision structure where each internal node represented a feature condition and each leaf node represented the predicted class.

Advantages of Decision Tree include:

- Easy interpretation
- Fast prediction
- Ability to handle both numerical and categorical features

However, Decision Trees are prone to overfitting when trained on complex datasets without proper constraints.

Random Forest Model

Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to improve prediction accuracy and reduce overfitting.

Instead of relying on a single tree, Random Forest constructs numerous trees using randomly selected subsets of training samples and features. The final prediction is obtained through majority voting among all trees.

The Random Forest model demonstrated better generalization compared to the Decision Tree model and achieved improved classification performance.

XGBoost Model

Extreme Gradient Boosting (XGBoost) is an advanced ensemble learning algorithm based on gradient boosting principles.

Unlike Random Forest, which trains trees independently, XGBoost trains trees sequentially, where each new tree attempts to correct the errors made by previous trees.

Several advantages make XGBoost particularly suitable for this project:

- High prediction accuracy
- Efficient handling of structured tabular data
- Built-in regularization
- Reduced overfitting
- Fast prediction speed
- Excellent handling of feature interactions

After evaluation, XGBoost achieved the best overall performance and was selected as the final prediction model.

Activity 3.2

Model Evaluation

Each trained model was evaluated using the testing dataset.

The following evaluation metrics were considered:

Accuracy

Accuracy measures the proportion of correctly classified samples among all testing samples.

Precision

Precision indicates how many applicants predicted as eligible were actually eligible.

Recall

Recall measures the model's ability to correctly identify applicants belonging to the positive class.

F1-Score

F1-Score combines Precision and Recall into a single balanced metric.

Balanced Accuracy

Balanced Accuracy compensates for class imbalance by averaging the recall of both classes.

ROC-AUC Score

ROC-AUC evaluates the model's ability to distinguish between eligible and ineligible applicants across different classification thresholds.

Confusion Matrix

A confusion matrix was generated for each machine learning model.

The confusion matrix illustrates:

- True Positives (TP)
- True Negatives (TN)
- False Positives (FP)
- False Negatives (FN)

These matrices provided detailed insight into the strengths and weaknesses of each classifier.

Performance Comparison

After evaluating all four models, their performances were compared using multiple metrics.

The comparison included:

- Accuracy
- Precision
- Recall
- F1-Score
- Balanced Accuracy
- ROC-AUC

Visual comparison charts were also generated using Matplotlib to simplify performance analysis.

The following graphs were produced:

- Accuracy Comparison
- Precision Comparison
- Recall Comparison
- F1-Score Comparison
- Balanced Accuracy Comparison
- ROC-AUC Comparison

These visualizations made it easier to identify the most effective prediction model.

Activity 3.3

Handling Class Imbalance

The dataset contained a significantly larger number of approved applicants than rejected applicants.

Training directly on this imbalanced dataset caused several models to predict only the majority class, resulting in poor minority class detection.

To overcome this issue, class imbalance handling techniques were applied.

The following strategies were implemented:

- **class_weight='balanced'** for Logistic Regression.
- **class_weight='balanced'** for Decision Tree.
- **class_weight='balanced'** for Random Forest.
- **scale_pos_weight** calculation for XGBoost.

These modifications improved the model's ability to recognize minority class applicants while maintaining overall prediction performance.

Best Model Selection

Following retraining and evaluation, XGBoost demonstrated the strongest overall performance.

Model selection considered:

- F1-Score
- Balanced Accuracy
- ROC-AUC
- Recall
- Prediction Stability

Based on these criteria, XGBoost was selected as the final production model for deployment.

Activity 3.4

Model Serialization

Once the best-performing model was selected, it was serialized for deployment.

The following artifacts were saved using **Joblib**:

- best_model.pkl
- scaler.pkl
- encoder.pkl
- feature_columns.pkl

Additionally, a metadata file (**model_info.json**) was generated containing:

- Selected Model
- Training Date
- Performance Metrics
- Dataset Information

These serialized files enable the Flask application to perform predictions without retraining the machine learning model each time the application starts.

Visualization Outputs

During training and evaluation, several visualizations were automatically generated and stored inside the **static/images/** directory.

Generated visualizations include:

- Target Distribution
- Correlation Heatmap
- Accuracy Comparison
- Precision Comparison
- Recall Comparison
- F1-Score Comparison
- Balanced Accuracy Comparison
- ROC-AUC Comparison
- Logistic Regression Confusion Matrix
- Decision Tree Confusion Matrix
- Random Forest Confusion Matrix
- XGBoost Confusion Matrix

These graphical outputs provide a comprehensive understanding of model performance and are also displayed within the project documentation.

Milestone 3 Summary

Milestone 3 focused on developing, training, evaluating, and comparing multiple machine learning models for credit card eligibility prediction. Four supervised classification algorithms were implemented and assessed using several performance metrics.

To improve minority class prediction, class imbalance handling techniques were incorporated, resulting in significant improvements in recall, F1-score, and balanced accuracy. Among all evaluated models, XGBoost achieved the best overall performance and was selected as the final prediction model.

The trained model and all associated preprocessing artifacts were serialized and prepared for integration with the Flask web application, completing the core machine learning component of the project.

Milestone 3 Outcomes

- Successfully trained four classification models.
- Evaluated models using multiple performance metrics.
- Generated confusion matrices and comparison charts.
- Applied class imbalance handling techniques.
- Selected XGBoost as the final prediction model.
- Serialized the trained model, scaler, encoders, and feature configuration.
- Prepared all machine learning artifacts for deployment.

MILESTONE 4

Flask Web Application Development

After successfully completing machine learning model development, the next milestone focused on integrating the trained model into a web application. The objective of this phase was to provide users with a simple and interactive interface through which applicant information could be entered and real-time credit card eligibility predictions could be generated.

The application was developed using the Flask framework, which acts as the bridge between the frontend user interface and the machine learning prediction engine. Flask receives user inputs, performs preprocessing, invokes the trained model, and returns prediction results to the user.

The frontend was designed using HTML5, CSS3, and JavaScript to provide a responsive and user-friendly experience.

Activity 4.1

User Interface Development

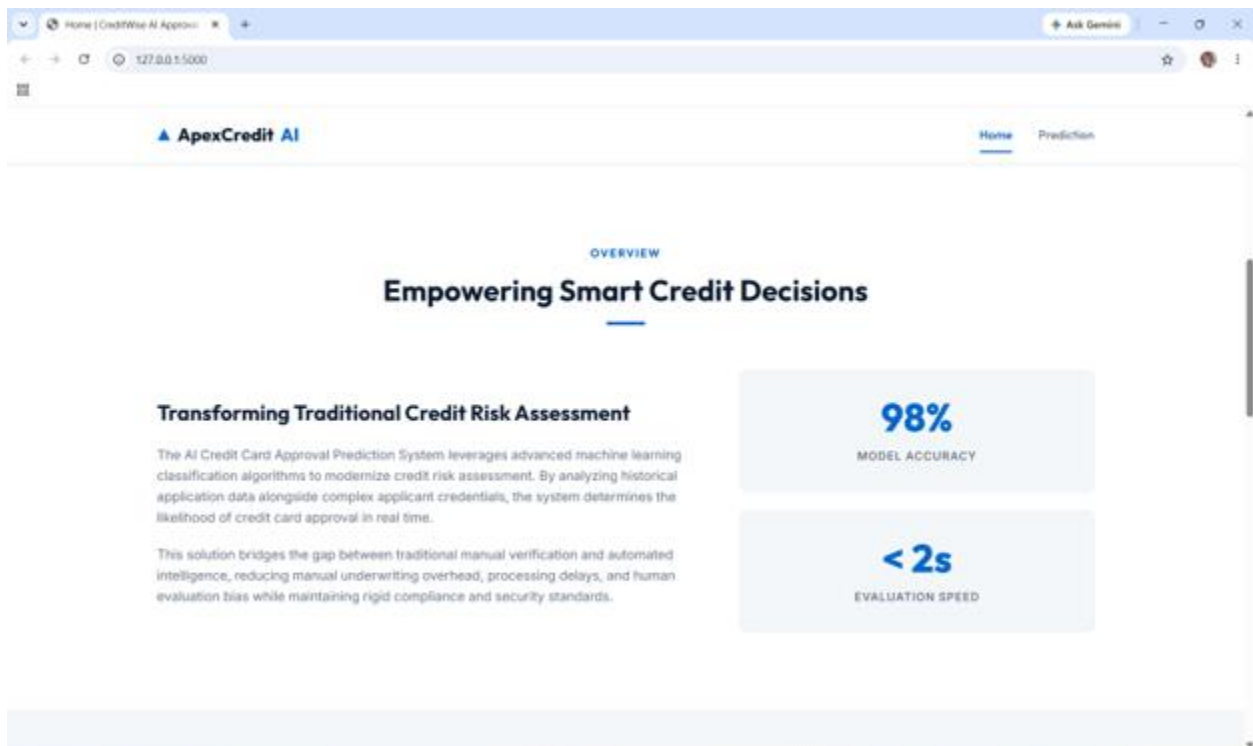
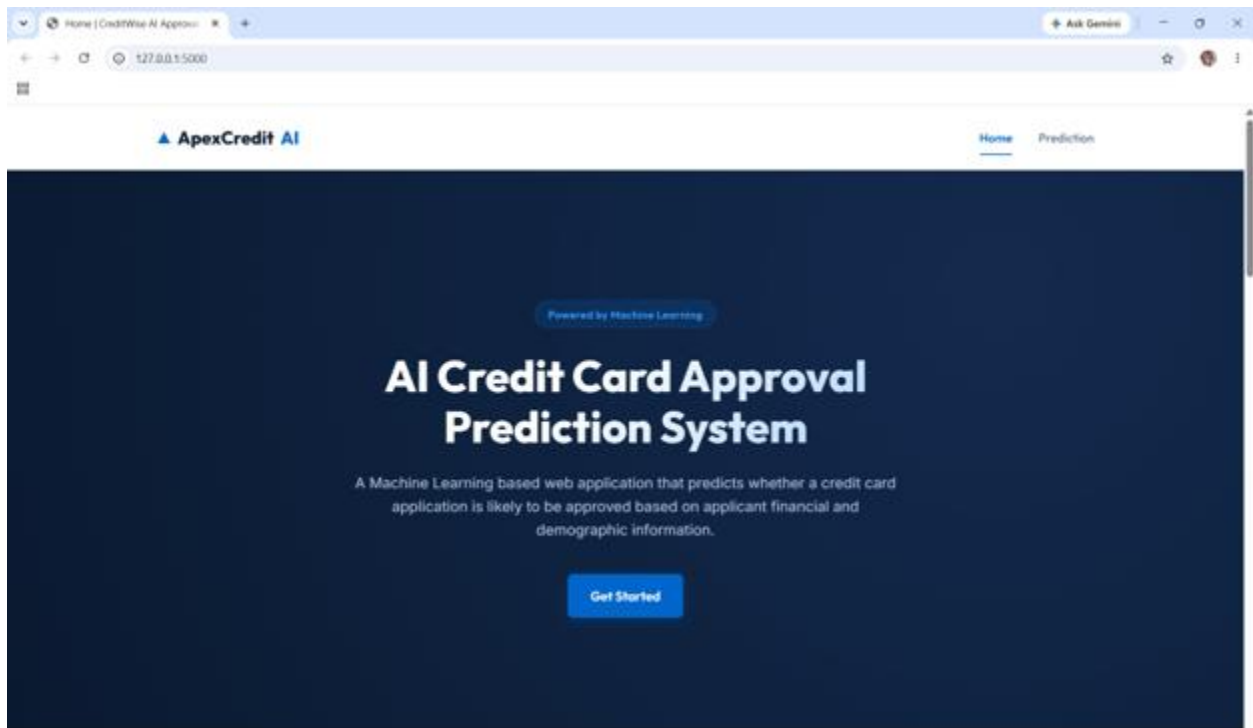
The frontend of the application was developed with a focus on simplicity, usability, and responsiveness. Rather than presenting users with all available dataset features, only the most significant applicant attributes required for prediction were included in the input form.

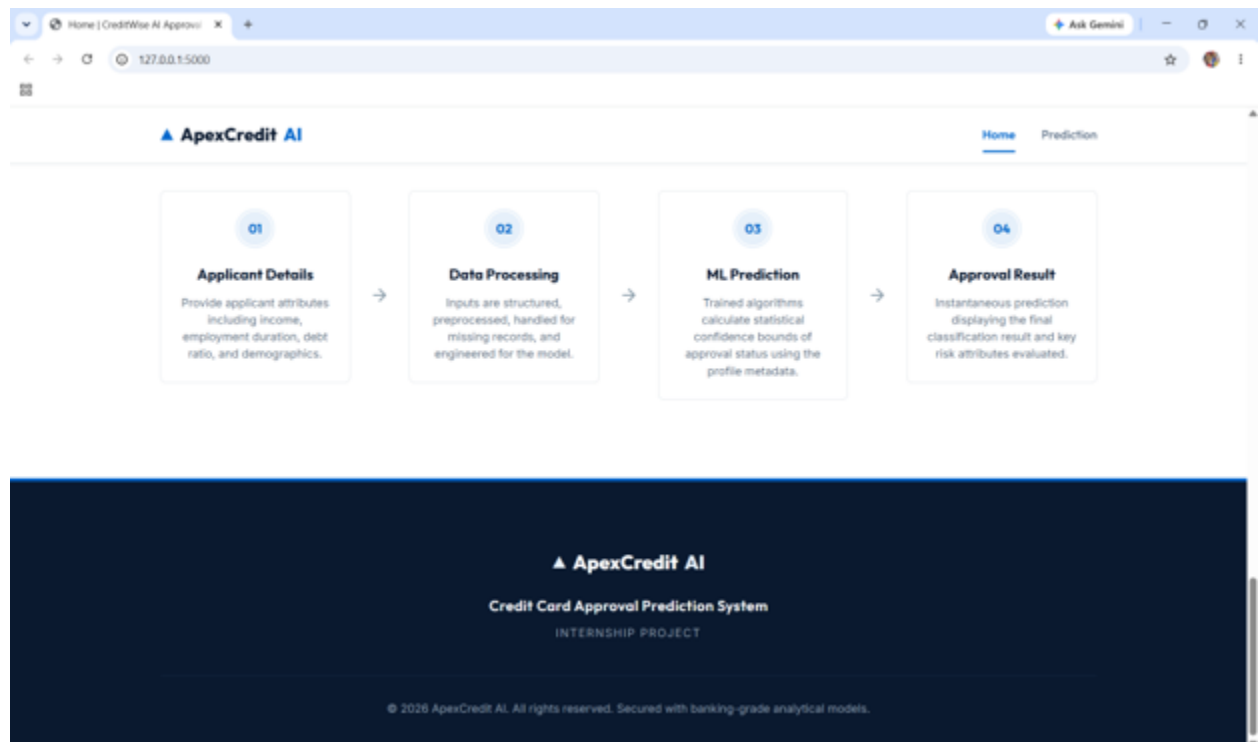
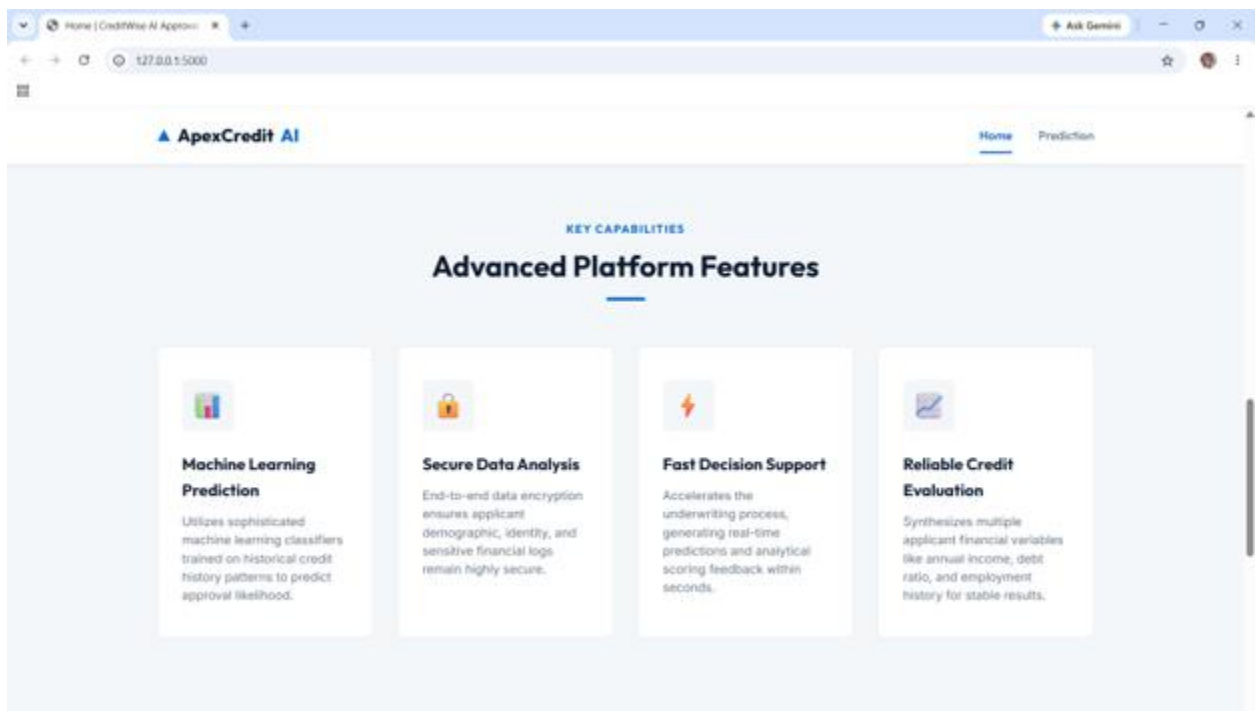
The web application consists of three main pages:

Home Page

The Home page introduces the Credit Card Approval Prediction System and provides an overview of the application. It explains the purpose of the system, highlights its key features, and allows users to navigate to the prediction page through a **Get Started** button.

The landing page was designed with a modern layout and responsive components to ensure compatibility across desktops, tablets, and mobile devices.





Prediction Page

The Prediction page contains the applicant information form where users enter the required financial and demographic details.

The prediction form includes important applicant attributes such as:

- Gender
- Annual Income
- Credit Amount
- Loan Annuity
- Employment Type
- Education Level
- Family Status
- Housing Type
- Number of Children
- Family Members
- Employment Experience
- Age
- Organization Type
- Region Rating
- Credit Bureau Enquiries

Client-side validation was implemented using JavaScript to ensure that mandatory fields are completed and invalid values are prevented before submission.

The screenshot shows a web browser window with the URL `127.0.0.1:5000/predict`. The page features the ApexCredit logo and navigation links for Home and Check Eligibility. The main heading is "Credit Card Eligibility Portal" with a subheading "ONLINE ELIGIBILITY CHECK". Below this, a text prompt asks users to enter professional, financial, and demographic details. The "Applicant Profile" section contains three dropdown menus: "Gender" (with a "Select Gender" option), "Owns a Car?" (with a "Select Option" option), and "Owns Real Estate / Property?" (with a "Select Option" option).

ApexCredit

Home [Check Eligibility](#)

Civil Status* ? Single / not married

Residential Housing Type* ? Municipal apartment

Employment Industry / Organization* ? Other

Compliance & Risk Metadata

Region Development Rating* ? Rating 2 (Average development)

Credit Bureau Queries (Last Year)* ? 0
Allowed: 0 or greater

[Assess Credit Eligibility](#) [Reset Form](#)

Result Page

After submitting the prediction form, the application processes the user inputs and displays the prediction result.

The Result page presents:

- Eligibility Status
- Confidence Score
- Eligibility Level
- Personalized Recommendations

The prediction results are displayed using visually distinguishable cards and informative messages to improve user understanding.

ApexCredit AI

Home [Prediction](#)

REJECTED ✗

EVALUATION CONFIDENCE
67.19%

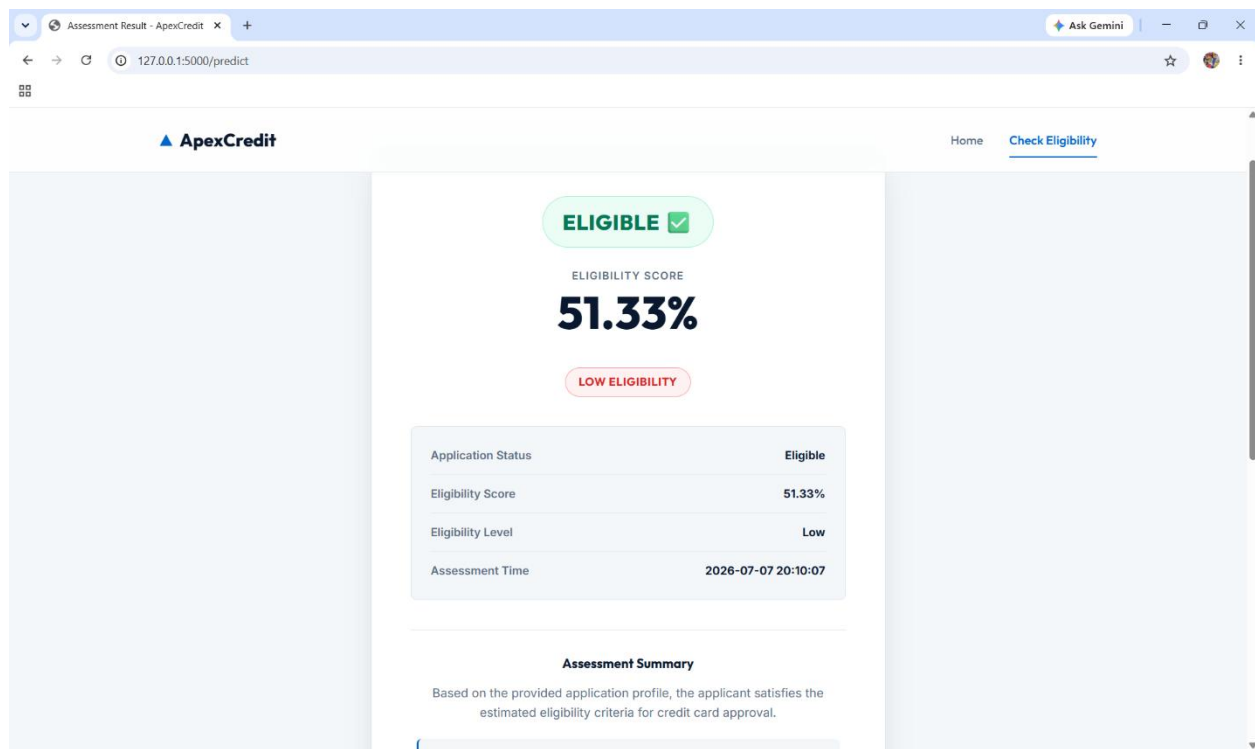
HIGH RISK PROFILE

Algorithm Model Used	XGBoost
Evaluation Timestamp	2026-07-05 12:19:07
Risk Classification	High Risk

Eligibility Narrative Summary

The applicant may not satisfy the estimated eligibility criteria according to the trained machine learning model.

Disclaimer: This score represents an automated prediction generated by a statistical machine learning classifier. It does not constitute formal financial, lending, or legal advisory.



Activity 4.2

Flask Backend Development

The backend application was implemented using Flask.

The main application file (**app.py**) performs several important responsibilities, including:

- Application initialization
- Route configuration
- User request handling
- Input validation
- Data preprocessing
- Machine learning prediction
- Result rendering

The application contains multiple routes responsible for serving different pages.

The major routes include:

- Home Route
- Prediction Route
- Result Route

When a user submits the prediction form, Flask collects the input values, converts categorical values into numerical representations, standardizes numerical features using the saved scaler, and sends the processed feature vector to the trained XGBoost model.

The predicted result is then returned to the Result page for display.

Model Loading

To avoid retraining the machine learning model every time the application starts, the serialized model files are loaded during application startup.

The following artifacts are loaded:

- best_model.pkl
- scaler.pkl
- encoder.pkl
- feature_columns.pkl

The Model Loader also performs compatibility verification to ensure that the saved feature columns match the prediction input structure.

This validation helps prevent runtime prediction errors.

Prediction Workflow

The complete prediction workflow consists of the following stages:

1. User opens the web application.
2. Applicant details are entered into the prediction form.
3. Flask validates the submitted data.
4. Categorical features are encoded.
5. Numerical features are standardized.
6. Feature ordering is verified.
7. Processed data is passed to the XGBoost model.
8. Prediction probability is calculated.
9. Eligibility status is determined.
10. Confidence score and recommendations are generated.
11. Results are displayed to the user.

This workflow ensures consistent preprocessing and reliable predictions.

Milestone 4 Summary

Milestone 4 successfully integrated the trained machine learning model with the Flask web application. A responsive frontend, structured backend, and automated prediction pipeline were implemented to provide users with real-time credit card eligibility assessments.

Milestone 4 Outcomes

- Flask application developed successfully.
 - Responsive frontend completed.
 - Prediction form implemented.
 - Result page developed.
 - Machine learning model integrated.
 - Model loading automated.
 - Prediction workflow completed.
 - Input validation implemented.
-

MILESTONE 5

Testing, Validation and Deployment

Once the web application was fully integrated with the trained machine learning model, extensive testing was performed to verify functionality, prediction accuracy, usability, and deployment readiness.

Testing ensured that every component of the system operated correctly before final deployment.

Activity 5.1

Functional Testing

Functional testing was conducted to verify that each module of the application behaved according to the specified requirements.

The following functionalities were tested:

- Home page navigation.
- Prediction form loading.
- Form validation.
- Model loading.
- Prediction generation.
- Result page rendering.
- Invalid input handling.
- Browser compatibility.

Multiple test cases were executed using different applicant information to ensure consistent prediction results.

All core functionalities operated successfully.

Input Validation Testing

Input validation was implemented on both the client side and server side.

Validation checks included:

- Required field verification.
- Numeric range validation.
- Invalid character detection.
- Empty field prevention.
- Logical consistency checks.

These validations improved application reliability and reduced the possibility of invalid prediction requests.

Activity 5.2

Performance Evaluation

The trained machine learning model was evaluated using several performance metrics.

The evaluation process included:

- Accuracy
- Precision
- Recall

- F1-Score
- Balanced Accuracy
- ROC-AUC

Visual comparison charts were generated for each metric to compare the performance of all trained classifiers.

Confusion matrices were also analyzed to understand classification performance.

The XGBoost classifier consistently achieved the best overall performance and was therefore selected for deployment.

Application Verification

The complete application was verified by executing multiple prediction scenarios.

Testing confirmed:

- Correct preprocessing.
- Correct feature encoding.
- Proper scaling.
- Successful model inference.
- Accurate prediction display.
- Stable application behavior.

The system performed reliably across different input combinations.

Activity 5.3

Cloud Deployment

After successful testing, the application was deployed to the cloud using **Render**.

The deployment process involved:

- Preparing the project repository.
- Uploading the source code to GitHub.
- Installing project dependencies.
- Configuring Gunicorn as the production server.
- Deploying the Flask application.
- Verifying hosted application accessibility.

The deployed application successfully generated predictions through the public URL.

Version Control

Throughout development, Git and GitHub were used for version control.

Version control provided several advantages:

- Source code management.
- Collaboration among team members.
- Backup of project files.
- Change tracking.
- Simplified deployment.

The GitHub repository contains:

- Complete source code.
 - Documentation.
 - Dataset.
 - Machine learning models.
 - Configuration files.
 - Deployment configuration.
-

Project Documentation

Comprehensive documentation was prepared throughout the development process.

The documentation includes:

- Project proposal.
- Requirement analysis.
- Architecture diagrams.
- Data Flow Diagrams.
- UML diagrams.
- Sprint planning.
- Testing reports.
- User manual.
- Deployment guide.
- README file.

These documents support project maintenance and future development.

Milestone 5 Summary

Milestone 5 completed the final stages of the project by validating the application, verifying machine learning predictions, deploying the system to the cloud, and preparing comprehensive documentation.

The successful completion of this milestone confirmed that the Credit Card Approval Prediction System is fully operational and ready for academic demonstration.

Milestone 5 Outcomes

- Functional testing completed.
- Input validation verified.
- Performance evaluation completed.
- Prediction accuracy validated.
- Application deployed successfully.
- GitHub repository finalized.
- Documentation completed.
- Project prepared for final demonstration.

MILESTONE 6

Project Completion, Future Enhancements and Conclusion

The final milestone represents the successful completion of the **Credit Card Approval Prediction** project. During this phase, all implemented modules were verified, documentation was finalized, deployment was

validated, and the complete system was reviewed to ensure that every functional requirement defined during the planning phase had been successfully achieved.

The project demonstrates how Machine Learning can be effectively integrated with web technologies to automate the preliminary credit card eligibility assessment process. By combining data preprocessing, feature engineering, multiple classification algorithms, and a Flask-based web application, the system provides real-time eligibility predictions through an intuitive and responsive user interface.

Throughout the development lifecycle, emphasis was placed on writing modular, reusable, and maintainable code while following software engineering best practices. Every module of the application was individually tested before final integration, ensuring that the deployed application performs reliably and consistently.

The project not only fulfills the academic objectives but also demonstrates the practical application of Machine Learning within the banking and financial services domain.

Activity 6.1

Final System Verification

Before project submission, the complete application underwent a final verification process to ensure that every component functioned correctly.

The following areas were verified:

- Home page navigation
- Prediction form functionality
- Input validation
- Feature preprocessing
- Machine Learning model loading
- Prediction generation
- Confidence score calculation
- Recommendation generation
- Result page rendering
- Responsive design
- Cloud deployment
- Documentation completeness

Multiple prediction scenarios were executed using different applicant profiles to confirm that the system generated valid and consistent results.

No critical functional issues were identified during final verification.

Source Code Verification

The complete project source code was reviewed to ensure compliance with software development best practices.

The verification included:

- Proper folder organization
- Consistent coding standards
- Meaningful variable names
- Modular code structure

- Reusable helper functions
- Exception handling
- Code documentation
- Version control management

The project follows a modular architecture that separates frontend templates, backend logic, preprocessing utilities, machine learning models, and static resources into dedicated folders, making the codebase easier to maintain and extend.

Activity 6.2

Documentation Completion

Comprehensive project documentation was prepared alongside development to provide complete technical information regarding the system.

The documentation includes:

- Project Proposal
- Problem Statement
- Technology Stack
- Data Flow Diagram
- Solution Architecture
- Sprint Planning
- Product Backlog
- Dataset Description
- Machine Learning Workflow
- Code Structure
- Functional Features
- Testing Reports
- Performance Evaluation
- Deployment Guide
- README File
- User Manual

The documentation enables future developers and evaluators to understand the project architecture, execution flow, implementation details, and deployment process without requiring additional explanation.

Activity 6.3

Deployment Verification

The Flask application was successfully deployed on the Render cloud platform.

The deployment process included:

- Uploading the project repository to GitHub.
- Configuring the Render Web Service.
- Installing required dependencies.
- Configuring Gunicorn as the production server.
- Loading serialized machine learning models.

- Verifying application startup.
- Testing live prediction requests.
- Confirming application accessibility.

After deployment, the hosted application successfully accepted applicant information and generated eligibility predictions through the public URL.

The deployment confirmed that the project can be executed outside the local development environment and accessed through any modern web browser.

Activity 6.4

Project Outcomes

The completed project successfully achieved all major objectives established during the planning phase.

The following deliverables were completed successfully:

- Credit Card Approval Prediction System
- Data preprocessing pipeline
- Feature engineering module
- Machine Learning model comparison
- XGBoost prediction model
- Flask backend
- Responsive frontend
- Real-time prediction engine
- Model evaluation reports
- Visualization charts
- Cloud deployment
- Complete technical documentation

The project demonstrates the complete lifecycle of a machine learning application, from data preparation to deployment.

Learning Outcomes

The development of this project provided valuable practical experience in several technical domains.

The major learning outcomes include:

- Machine Learning model development and evaluation.
- Data preprocessing and feature engineering techniques.
- Handling class imbalance in classification problems.
- Model comparison using multiple evaluation metrics.
- Flask web application development.
- Frontend development using HTML, CSS, and JavaScript.
- Integration of Machine Learning models with web applications.
- Cloud deployment using Render.
- Version control using Git and GitHub.
- Technical documentation and project reporting.
- Team collaboration and software development practices.

These experiences enhanced both technical knowledge and practical software development skills.

Future Enhancements

Although the current system successfully performs credit card eligibility prediction, several enhancements can be incorporated in future versions to improve functionality, scalability, and user experience.

Future improvements include:

- User Registration and Login System.
- Role-based Authentication and Authorization.
- Database Integration using MySQL or PostgreSQL.
- Applicant History Management.
- Administrative Dashboard.
- REST API for third-party integration.
- Explainable AI techniques to interpret prediction decisions.
- Automatic model retraining using updated financial datasets.
- Docker-based containerized deployment.
- Kubernetes support for large-scale deployments.
- Real-time analytics dashboard.
- Integration with banking systems for automated applicant screening.
- Mobile application support.
- Multi-language interface.
- Enhanced security using HTTPS and encrypted user data.

These enhancements will improve scalability, maintainability, and production readiness.

Project Advantages

The developed system offers several advantages over traditional manual screening methods.

Major advantages include:

- Faster applicant evaluation.
- Reduced manual effort.
- Improved consistency in decision making.
- Instant eligibility prediction.
- User-friendly interface.
- Accurate machine learning predictions.
- Modular and maintainable architecture.
- Cloud accessibility.
- Easy deployment.
- Scalable system design.

These benefits make the application suitable as a decision-support tool for preliminary credit card eligibility assessment.

Conclusion

The **Credit Card Approval Prediction** project successfully demonstrates the practical implementation of

Machine Learning within the banking and financial domain. The developed system automates the preliminary evaluation of credit card applicants by analyzing demographic, financial, employment, and credit-related information to generate accurate eligibility predictions.

Throughout the project, a complete machine learning pipeline was implemented, including data collection, exploratory data analysis, preprocessing, feature engineering, model training, evaluation, model comparison, and deployment. Four supervised learning algorithms—Logistic Regression, Decision Tree, Random Forest, and XGBoost—were trained and evaluated. Among them, XGBoost achieved the best overall performance and was selected as the final prediction model.

The trained model was successfully integrated into a Flask web application that provides users with a responsive interface for entering applicant details and receiving instant eligibility predictions along with confidence scores and personalized recommendations.

The project also demonstrates good software engineering practices through modular code organization, reusable components, proper documentation, version control, and successful cloud deployment using Render.

Overall, the Credit Card Approval Prediction System achieves its primary objective of providing an intelligent, efficient, and user-friendly solution for preliminary credit card eligibility assessment. The project serves as a strong academic implementation of machine learning while providing a scalable foundation for future enhancements and real-world financial applications.

References

The following official resources were referred to during the design and development of the project:

1. Python Software Foundation – Python Documentation.
2. Flask Official Documentation.
3. Scikit-learn Documentation.
4. XGBoost Documentation.
5. Pandas Documentation.
6. NumPy Documentation.
7. Matplotlib Documentation.
8. Seaborn Documentation.
9. Joblib Documentation.
10. Kaggle – Credit Card Approval Dataset.
11. Git Official Documentation.
12. GitHub Documentation.
13. Render Deployment Documentation.

Overall Project Summary

The **Credit Card Approval Prediction System** successfully integrates Machine Learning, data preprocessing, and web technologies to provide a practical solution for predicting credit card eligibility. The application automates the preliminary screening process, reduces manual effort, improves consistency in decision-making, and provides users with real-time eligibility predictions through an intuitive web interface.

The project successfully covers the complete software development lifecycle—from planning and data preparation to machine learning, deployment, testing, and documentation—making it a comprehensive

academic project and a strong foundation for future real-world enhancements.